AutoChip Tutorial Assignment Report

LLM4ChipDesign | Tanvir Shahriar

1. AutoChip parameters

Same for all runs. Model: `nvidia/nemotron-3-super-120b-a12b` (NVIDIA OpenAI-compatible endpoint).

```
"model_family": "ChatGPT", "num_candidates": 5, "iterations": 5, "ensemble": false
```

Testbenches: top module renamed to `tb`, errors counted, last line `Mismatches: N in M samples`. Verification of the best response:

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb -o <best>/<module>.vvp <best>/<module>.sv ./<name>_tb.sv
vvp -n <best>/<module>.vvp
```

2. Summary

Example	Iteration 0 ranks	Feedback rounds	Best response	Verification
Example 1 - binary_to_bcd	1, 1, 1, 1, −2	0	iter0/response2	0 mismatches / 32
Example 2 - sequence_detector	−1, −1, 0.83, 1, −0.5	0	iter0/response3	0 / 12
Example 3 - shift_register	0.25, 0.25, −2, 0.25, 0.25	1	iter1/response1	0 / 8
Example 4 - abro_state_machine	0.06, 0.06, 0.06, 0.06, −2	1	iter1/response2	0 / 16
Example 5 - dice_roller	−2, 1, −2, −2, 1	0	iter0/response1	0 / 4000
Example 6 - lfsr	0.004, −1, −1, 0.004, 0.004	1	iter1/response3	0 / 257
Example 7 - sequence_generator	1, 1, 0, 0, 0	0	iter0/response1	0 / 17
Example 8 - traffic_light	0.25, 1, 1, 1, 1	0	iter0/response2	0 / 4
8-bit CPU (cpu8_acc)	−2, −2, −2, −2, −2	5	iter5/response4	0 / 5

Rank: (samples − mismatches) / samples; −0.5 compile warning, −1 compile error, −2 no Verilog in response.

3. Examples

Example 1 - binary_to_bcd

Specification: 5-bit binary input → 8-bit BCD (tens, ones). Template `binary_to_bcd_converter(binary_input[4:0], bcd_output[7:0])`.

Trajectory: Iteration 0: 4 of 5 candidates passed; best selected, no feedback.

Verification: rank 1.0, `Mismatches: 0 in 32 samples`.

Example 2 - sequence_detector

Specification: Clock, active-low reset, 3-bit data; sequence_found on 001, 101, 110, 000, 110, 110, 011, 101.

Trajectory: Iteration 0: 2 compile errors, 1 compile warning, 1 with 2 mismatches, 1 passed; no feedback.

Verification: rank 1.0, `Mismatches: 0 in 12 samples`.

Example 3 - shift_register

Specification: Clock, active-low reset, 1-bit data, shift enable; 8-bit output.

Trajectory: Iteration 0: designs shifted `{data_out[6:0], data_in}` (rank 0.25). Feedback: mismatch list (e.g. `Expected: 10000000, Got: 00000001`). Iteration 1: `{data_in, data_out[7:1]}`, all passed.

Verification: rank 1.0, `Mismatches: 0 in 8 samples`.

Example 4 - abro_state_machine

Specification: Clock, active-low reset, inputs A, B; outputs O and one-hot State.

Trajectory: Iteration 0: wrong transitions (rank 0.0625). Feedback: failing cases with expected/actual O and State. Iteration 1: transitions rewritten, 4 of 5 passed.

Verification: rank 1.0, `Mismatches: 0 in 16 samples`. The final design only reaches DONE when B comes before A; the 16 test cases do not cover A before B.

Example 5 - dice_roller

Specification: Clock, active-low reset, 2-bit die select, roll; rolled number for a 4, 6, 8 or 20-sided die.

Trajectory: Iteration 0: 2 passed, 3 returned no Verilog; no feedback.

Verification: rank 1.0, `Mismatches: 0 in 4000 samples`.

Example 6 - lfsr

Specification: Clock, active-low reset; 8-bit output, initial state 10001010, taps at 1, 4, 6, 7.

Trajectory: Iteration 0: taps taken as 0-based bits (rank 0.004), 2 compile errors. Feedback: expected vs actual value each cycle. Iteration 1: 1-based taps, all passed.

Verification: rank 1.0, `Mismatches: 0 in 257 samples`.

Example 7 - sequence_generator

Specification: Clock, active-low reset, enable; repeats AF, BC, E2, 78, FF, E2, 0B, 8D while enabled.

Trajectory: Iteration 0: 2 passed, 3 ran one step ahead (rank 0); no feedback.

Verification: rank 1.0, `Mismatches: 0 in 17 samples`.

Example 8 - traffic_light

Specification: Clock, active-low reset, enable; red → green after 32 cycles, green → yellow after 20, yellow → red after 7.

Trajectory: Iteration 0: 4 passed, 1 had wrong transition timing (rank 0.25); no feedback.

Verification: rank 1.0, `Mismatches: 0 in 4 samples`.

4. Feedback causing an RTL change (Example 6 - lfsr)

```
iteration 0 (rank 0.004):  data <= {data[6:0], data[1] ^ data[4] ^ data[6] ^ data[7]};
feedback:                  Error: At time 20000, data = 00010100, expected = 00010101  (...)
iteration 1 (rank 1.0):    data <= {data[6:0], data[0] ^ data[3] ^ data[5] ^ data[6]};
```

The specification does not say how tap locations are numbered. In iteration 0 the compiling candidates used them as 0-based bit indices (256 of 257 mismatches). From the expected values in the feedback, all five iteration-1 candidates switched to 1-based tap locations and passed. Example 3 - shift_register and Example 4 - abro_state_machine were fixed the same way in one feedback round.

5. 8-bit CPU

Starting prompt, then one follow-up asking the model to settle on a final design (ISA with encodings, registers, memory, port list); log in `cpu8/codesign_chat.txt`:

```
Let us make a brand new microprocessor design together. We're severely constrained on space and I/O.
We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an
accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we
should begin?
```

Proposed design: 16 single-byte instructions (`[7:4]` opcode, `[3:0]` operand), 8-bit ACC, MAR, PC, IR and Z/N/C flags, separate 256×8 instruction ROM and data RAM, module `cpu8_acc`. The AutoChip prompt is the starting prompt plus this design and port list.

Testbench: one program (LOAD 3, OUT and count down to 0, ADD 9, STORE), 5 checks (number of OUTs, three OUT values, RAM[5]). On failure it prints the expected and actual instruction-address order and RAM writes.

Trajectory:

Iteration	Ranks	Step
0	−2 ×5	No Verilog returned.
1	−2 ×4, −1	Compile errors.
2–3	−1 ×5	Compile errors (declarations inside case branches, bit-select of expressions).
4	−1 ×4, 0.2	First simulating design: instruction latched in an IR register, jumps one address late, no STORE.
5	0.2, 1 ×4	IR removed, instruction decoded directly from `imem_data` (single-cycle); passed.

A first run, before the execution trace was added to the testbench, reached best rank 0.0 after 4 iterations and stopped on an API timeout; its log is `cpu8/log_first_run_not_final.txt`.

Result: rank 1.0, `Mismatches: 0 in 5 samples` (best response `iter5/response4`). The design also adds a hardware stack pointer for CALL/RET.

Cell count (Yosys): 477 cells, including 34 flip-flops, within the 1000-cell budget.

```
yosys -p "read_verilog -sv outputs_cpu8/iter5/response4/cpu8_acc.sv; synth -top cpu8_acc;
          abc -g AND,NAND,OR,NOR,XOR,XNOR,MUX; stat"
```

Remaining limitations:

- AND, OR, XOR, JC, IN, CALL/RET and the N flag are not tested.
- Works only with an instruction ROM that is read in the same cycle, not with a synchronous ROM.
- The cell count is for Yosys generic gates, not a real standard-cell library.